



Bài 4

TƯ DUY CÔNG NGHỆ NỀN TẢNG & ĐÓNG GÓI SERVICE VỚI DOCKER

Giảng viên: ThS. Nguyễn Văn Nhân

Email: nhannv@dainam.edu.vn

Điện thoại: 0346542854

1. Giải thích được tại sao Container là nền tảng của DevEx và Platform Engineering
2. Viết được file Dockerfile cho service của nhóm
3. Build và chạy được Image lên Registry với lớp có tag quy ước



1. Đặt vấn đề
2. Công nghệ nền tảng
3. Docker
4. Thực hành

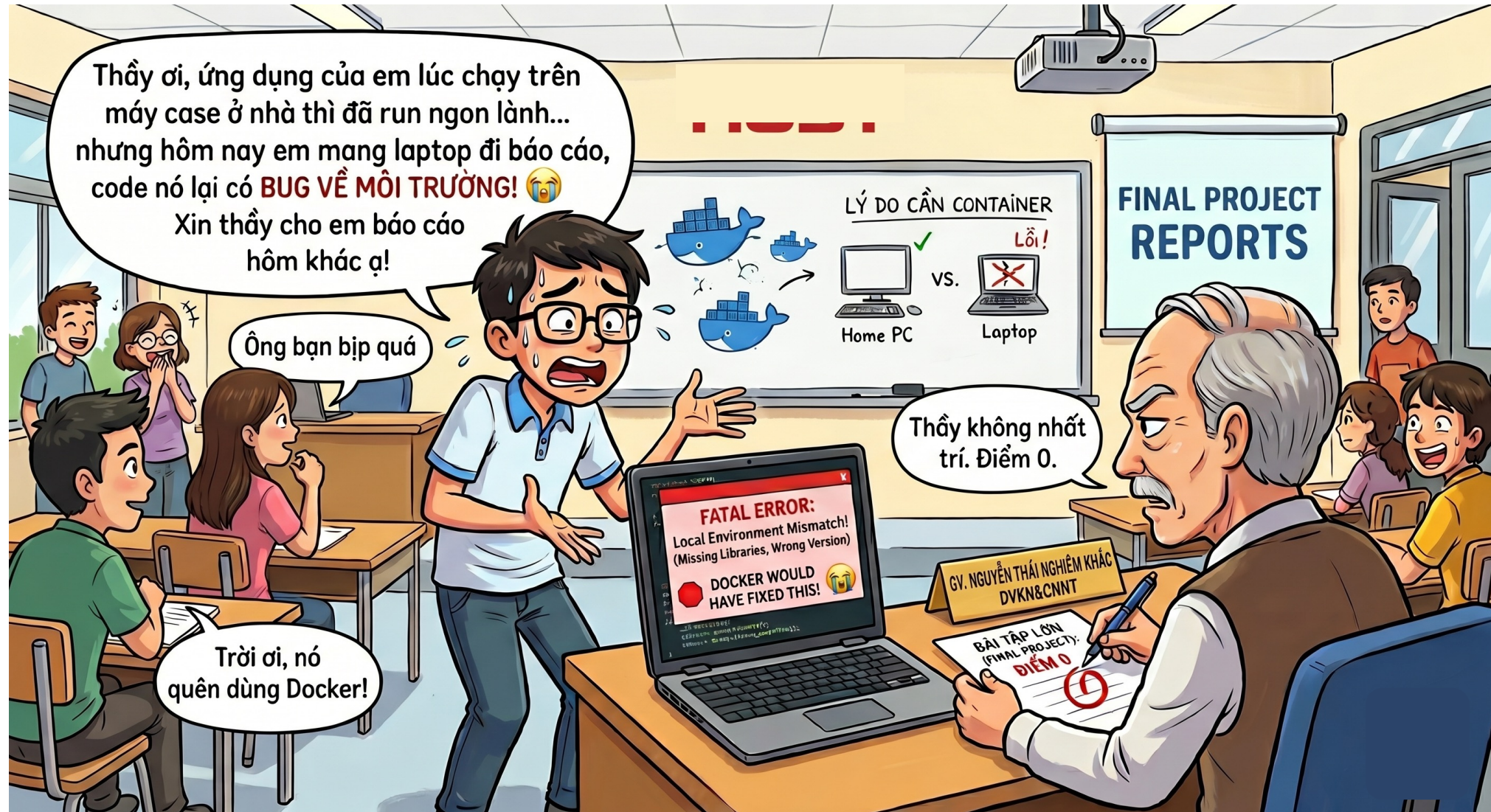


ĐẶT VẤN ĐỀ



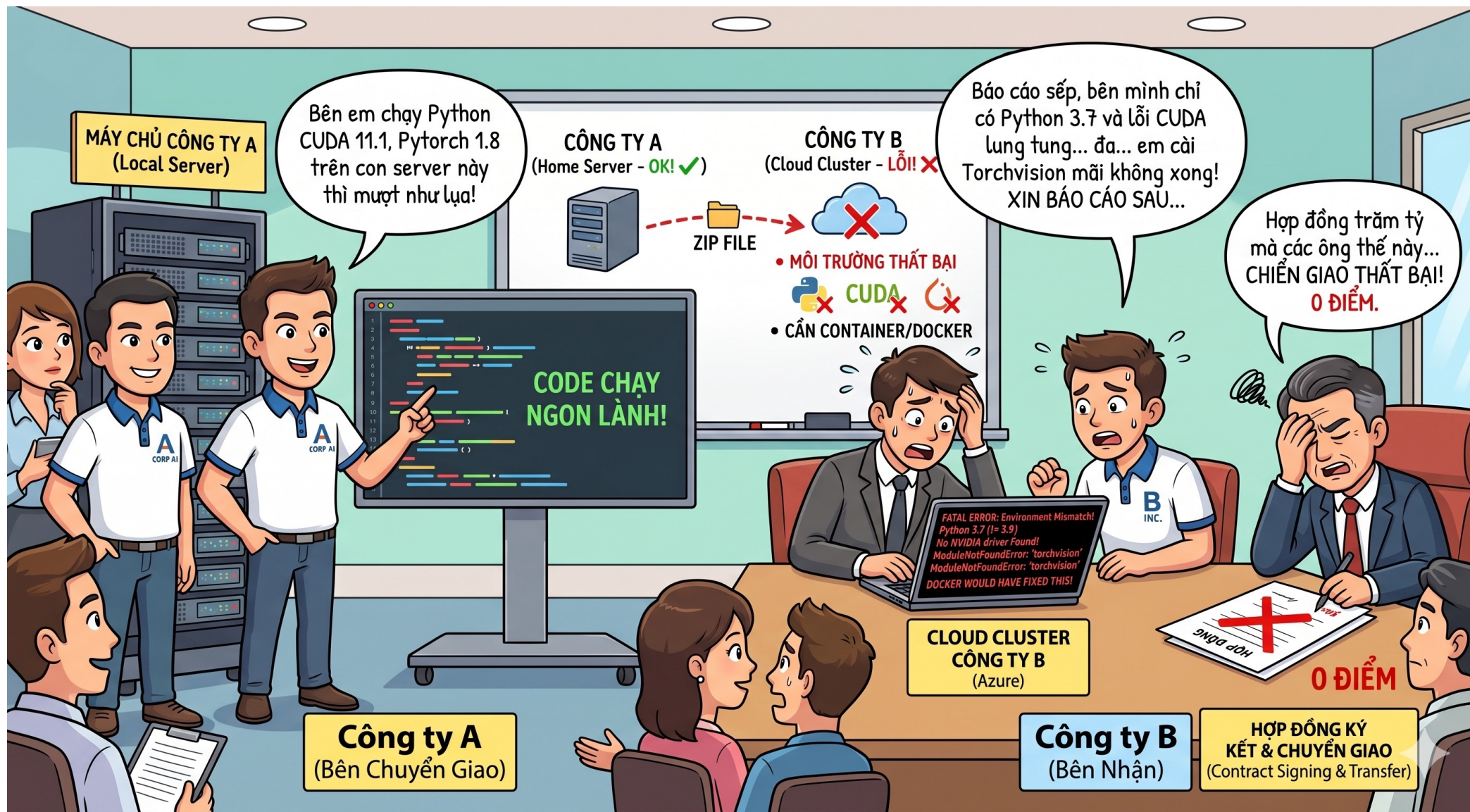
ĐẶT VẤN ĐỀ

Trong một buổi báo cáo bài tập lớn nọ:



ĐẶT VẤN ĐỀ

Chuyển giao công nghệ giữa hai công ty:

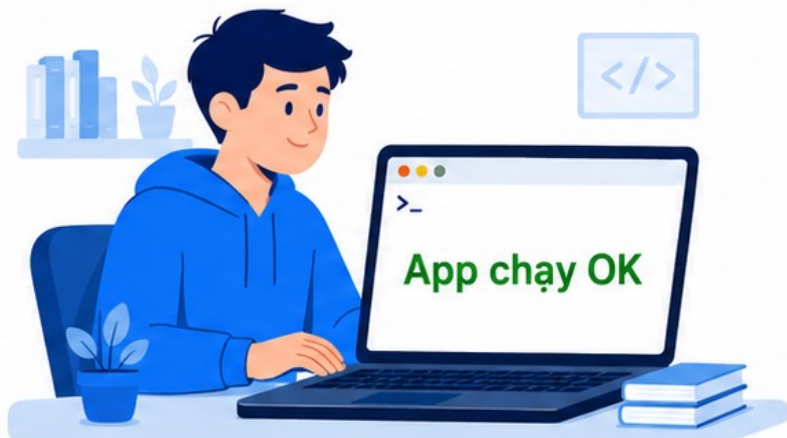


IT WORKS ON MY MACHINE?

Vì sao Docker ra đời?



Máy em chạy được



source code



Khác môi trường



Máy khác lỗi



Sai phiên bản
Node/Python



Thiếu thư viện



Thiếu .env



Khác hệ điều hành



Sai cấu hình



Cùng source code nhưng môi trường khác nhau
có thể cho kết quả khác nhau.



Docker giúp đóng gói môi trường
chạy thống nhất.



CÔNG NGHỆ NỀN TẢNG



SAI LẦM TRỊ GIÁ 200 TỶ USD

Bài học từ Blockbuster & Netflix



N Thách thức của Netflix

1 Người dùng tăng nhanh



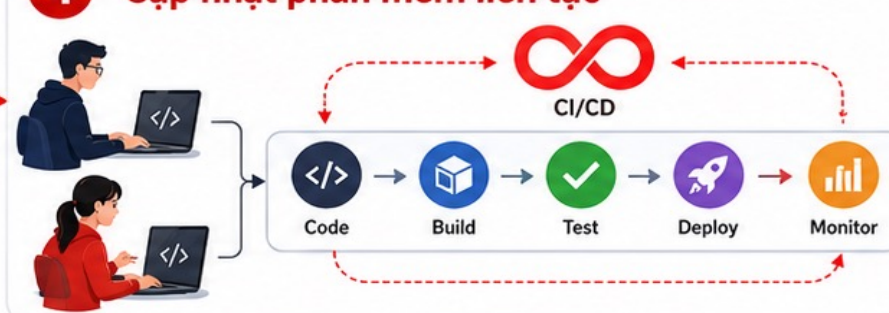
2 Hệ thống quá lớn



3 Video phải mượt toàn cầu



4 Cập nhật phần mềm liên tục



5 Hệ thống có thể lỗi bất kỳ lúc nào



6 Dữ liệu và hành vi người dùng rất lớn







AWS Cloud



Auto Scaling



Microservices



Open Connect CDN



Spinnaker CI/CD



Monitoring



Resilience

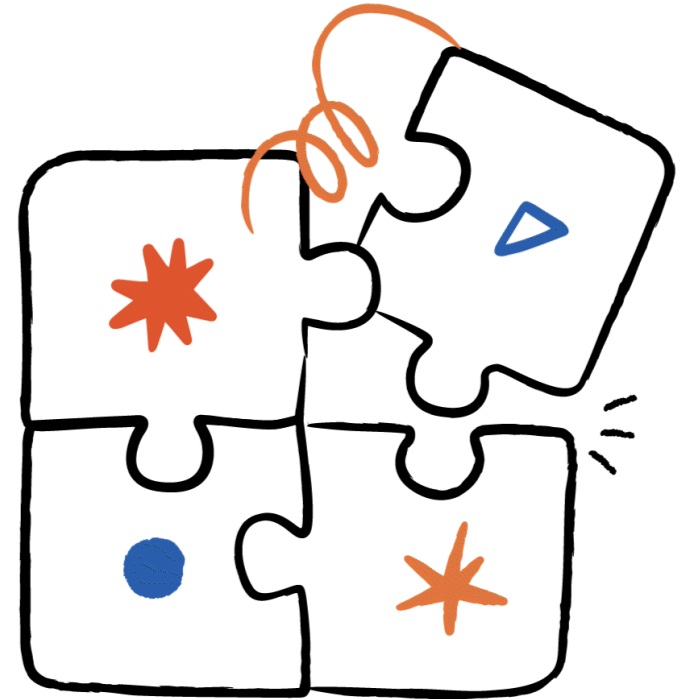


Chaos Monkey



Analytics Platform

- ❖ Công nghệ nền tảng (Platform Technology) trong phát triển phần mềm là tập hợp các bộ công cụ, hạ tầng, thư viện và kiến trúc tiêu chuẩn dùng để xây dựng, triển khai và vận hành các ứng dụng một cách tối ưu. Nó đóng vai trò như "móng nhà" giúp rút ngắn thời gian và đảm bảo tính ổn định cho dự án

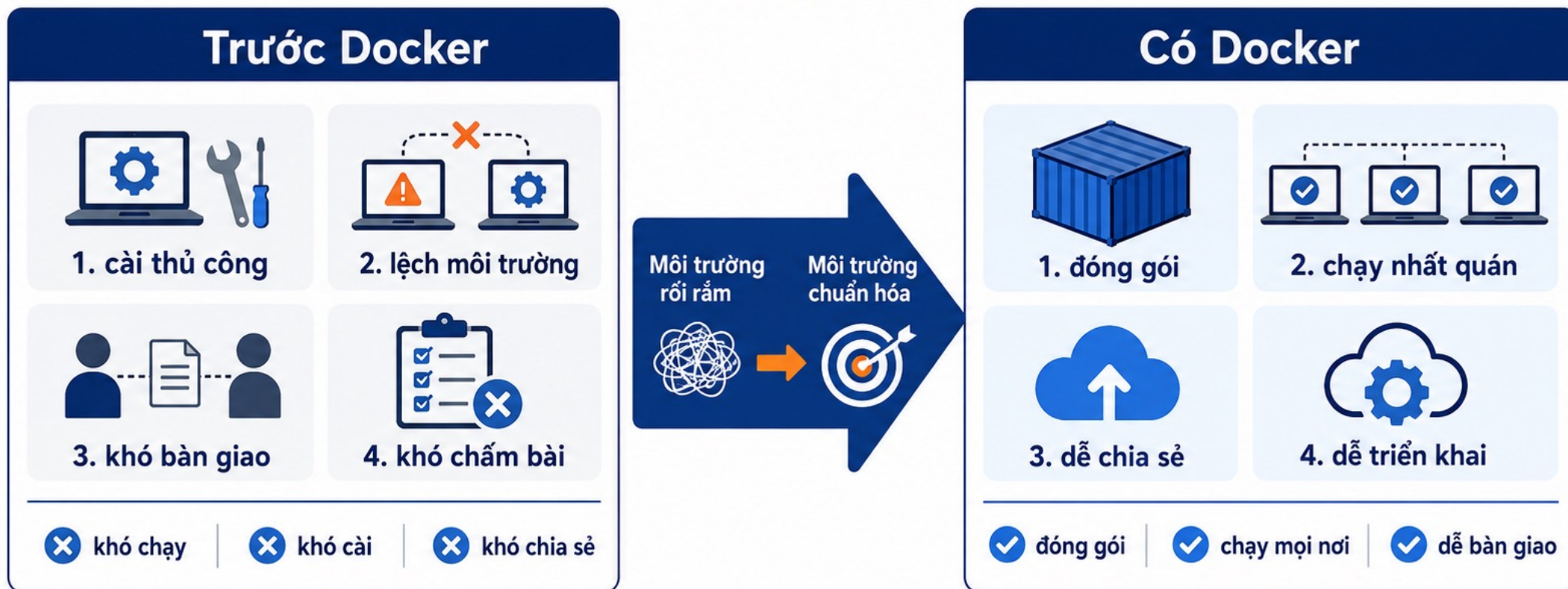


Key Types of Platform Technologies

- **Computing & OS Platforms:** Operating systems like Windows, iOS, or Linux manage hardware resources and provide the basic environment for users and apps.
- **Cloud Platforms:** Services like Amazon Web Services (AWS), Microsoft Azure, and Google Cloud allow businesses to access massive computing and storage infrastructure over the internet.
- **Application & API Platforms:** Frameworks that allow developers to build, test, and integrate their own custom software solutions or digital products easily.
- **Data & Analytics Platforms:** Databases and data management tools that organize, process, and extract insights from large datasets

DOCKER GIẢI QUYẾT ĐIỀU GÌ?

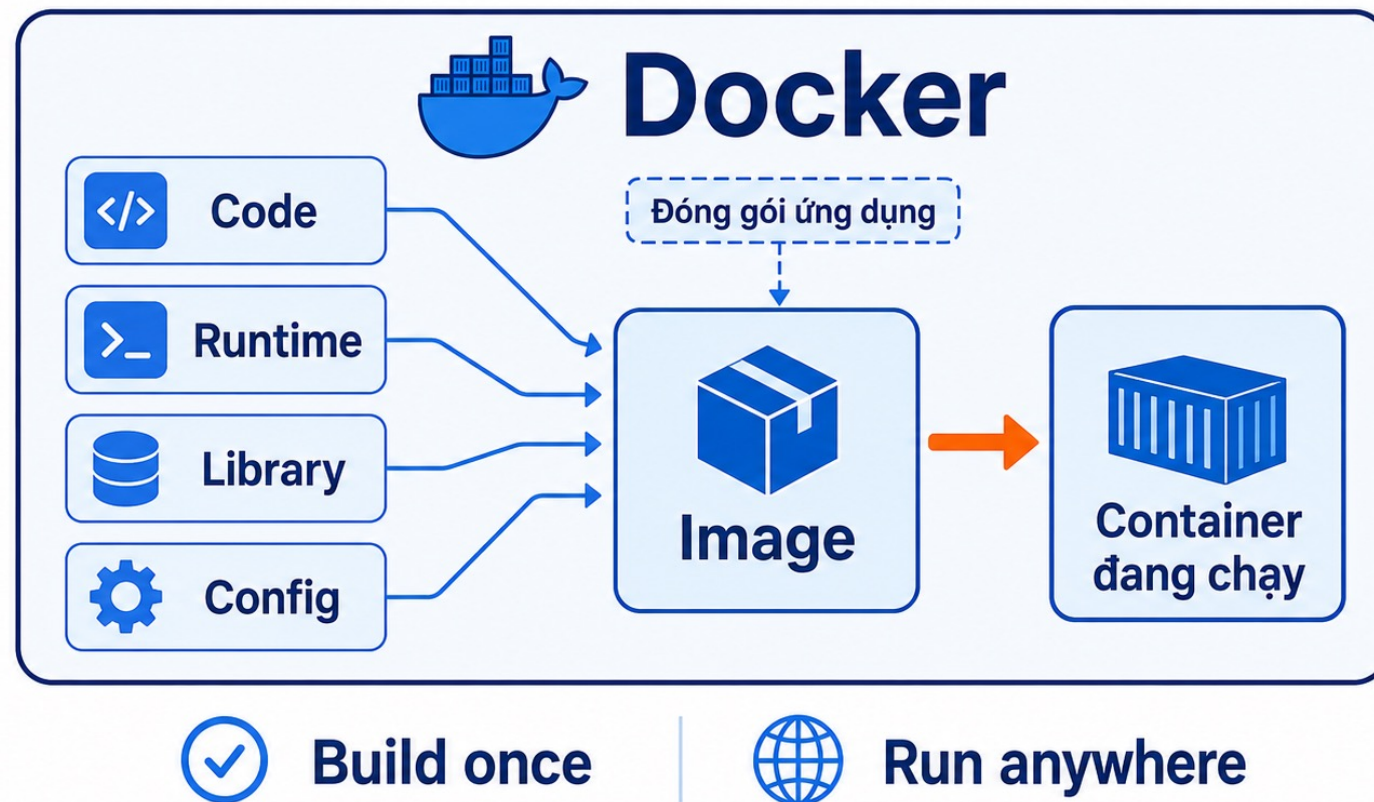
Chuẩn hóa môi trường phát triển và triển khai



Docker giúp đóng gói ứng dụng và môi trường để chạy nhất quán trên mọi máy.

DOCKER

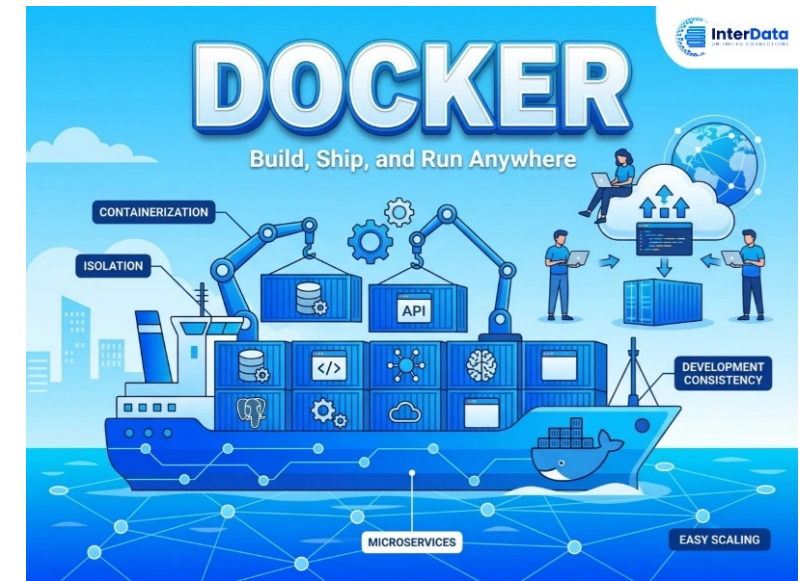
- ❖ Docker là công cụ đóng gói Code + Runtime + Library + Config vào 1 file gọi là IMAGE. Image đó có thể chạy trên bất kỳ máy nào có Docker cài sẵn — Mac, Windows, Linux, Cloud



DOCKER

❖ Với engineer:

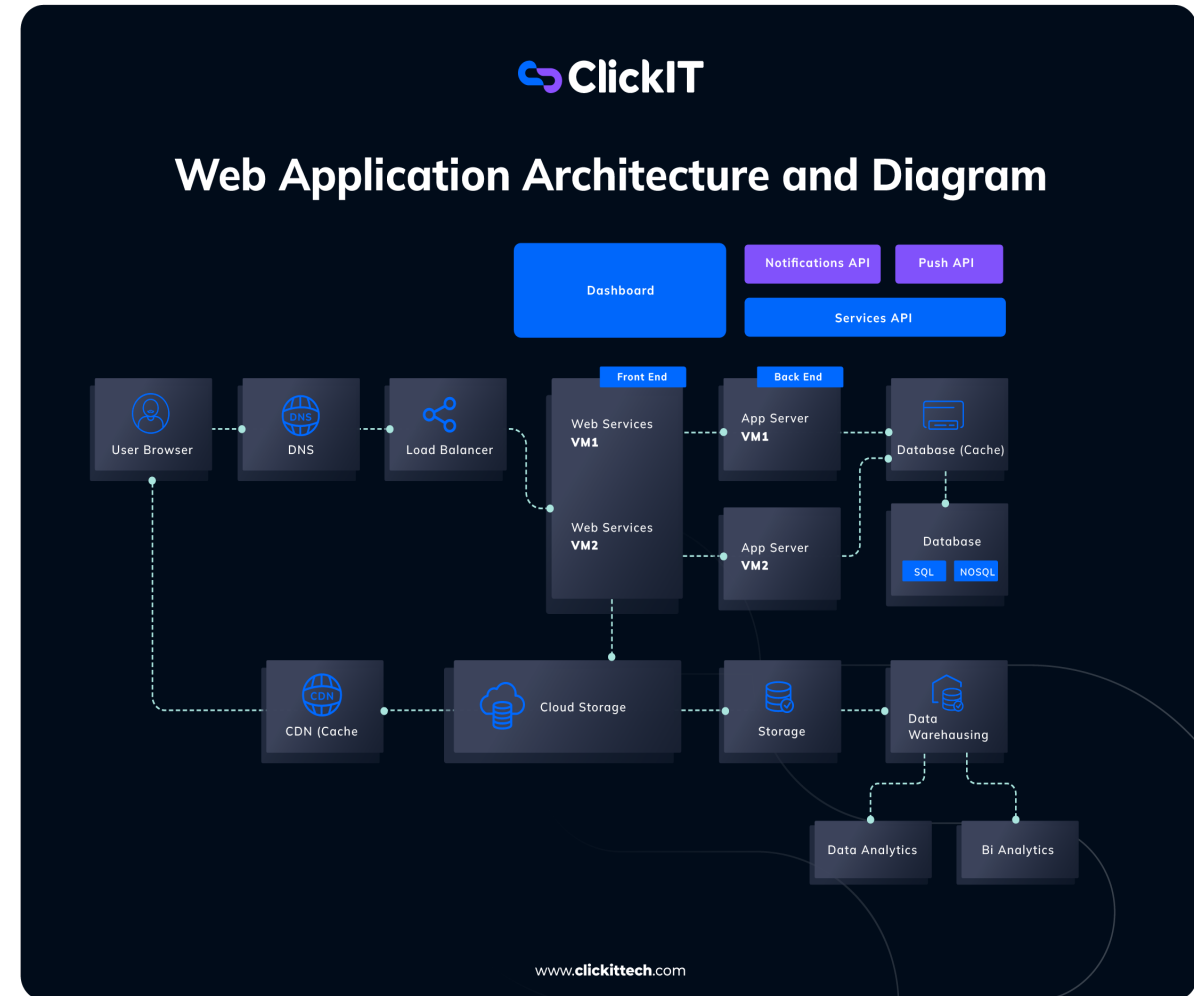
- **Build Once, Run Anywhere:** cho phép đóng gói Code + language(Python/NodeJS) + Thư viện + Cấu hình... vào một khối thống nhất gọi là **Image**, *có thể run trên các máy khác nhau:* Mac, Windows, Server Cloud
- **Tính độc lập:** có thể chạy Dự án A (dùng Python 2.7) và Dự án B (dùng Python 3.9) trên *cùng một cái laptop mà không gây xung đột*. Mỗi ứng dụng nằm trong một Container riêng biệt
- **Onboarding & Deployment:** Không cần thực hiện lại quá trình cài đặt môi trường, môi trường sẽ *dựng lên sẵn sàng chỉ cần một dòng lệnh*



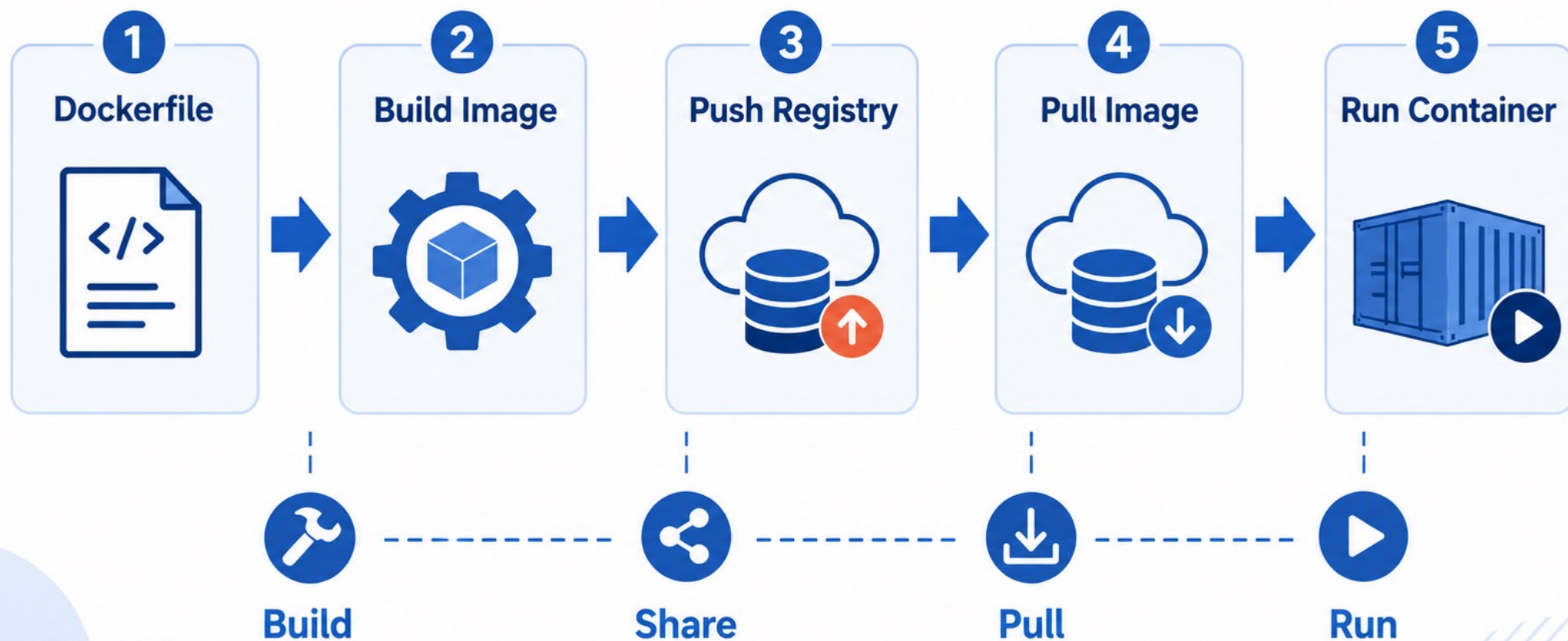
DOCKER

❖ Với tổ chức:

- Quản lý, phân phối và vận hành chúng ở quy mô, lưu lượng truy cập lớn trên nền tảng Cloud.
- Developer Experience: Giảm thiểu gánh nặng cấu hình, giúp các developer tập trung vào tối ưu code.
- CI/CD: tự động hoá hoàn toàn từ quá trình xây dựng đến triển khai.



CHU TRÌNH DOCKER

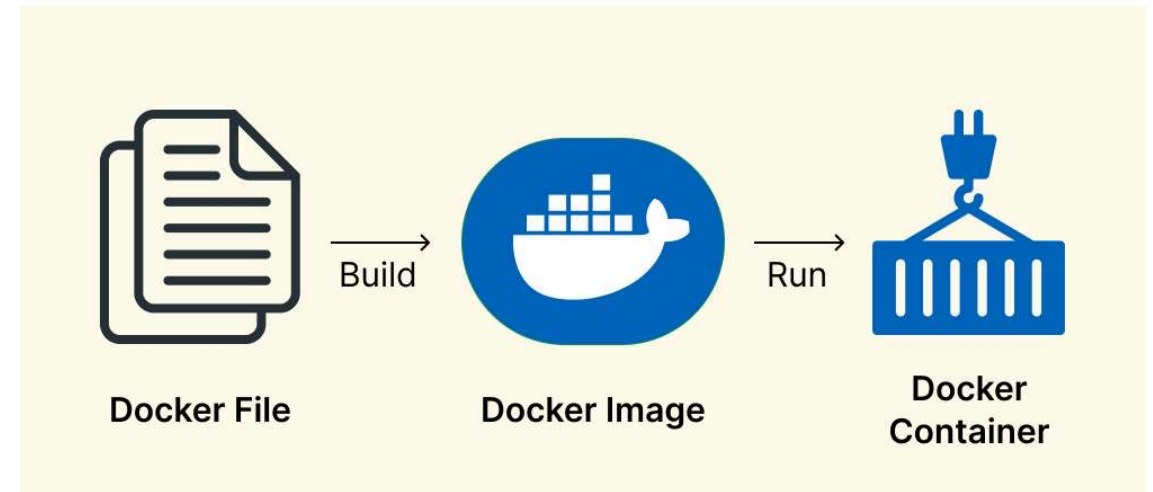


DOCKER FILE

- ❖ **Dockerfile** là một tệp văn bản mã nguồn đơn giản, không có phần mở rộng, chứa một tập hợp các chỉ thị tuần tự để tự động xây dựng một **Docker Image**.
- ❖ Giúp Docker biết chính xác **cần chuẩn bị** hệ điều hành, thư viện gì command thứ tự chạy ứng dụng.

```
FROM node:node:14-alpine
MAINTAINER AnTien<antiendhcnhn@gmail.com>
WORKDIR /app
EXPOSE 3000
COPY package.json /app
RUN npm install
COPY . /app
CMD ["npm", "run", "start"]
```

Dockerfile



Quy trình tạo Docker Container

DOCKERFILE SYNTAX

Cấu trúc Dockerfile mẫu cơ bản

1. Chọn Image nền có sẵn Node.js phiên bản 18 phiên bản Alpine

FROM node:18-alpine

2. Tạo và chuyển hướng vào thư mục làm việc bên trong container

WORKDIR /app

3. Sao chép file danh sách thư viện vào trước để tận dụng Docker Cache

COPY package*.json ./

4. Chạy câu lệnh cài đặt các thư viện cần thiết

RUN npm install

5. Sao chép toàn bộ mã nguồn còn lại từ máy bạn vào thư mục /app

COPY . .

6. Thông báo container sẽ chạy ở cổng 3000

EXPOSE 3000

7. Lệnh khởi chạy ứng dụng khi container bắt đầu chạy

CMD ["npm", "start"]

ĐỌC HIỂU CÁC LỆNH DOCKERFILE

1

FROM



chọn image nền

2

WORKDIR



thư mục làm việc

3

COPY



chép source/dependency

4

RUN



cài thư viện

5

EXPOSE



cổng ứng dụng

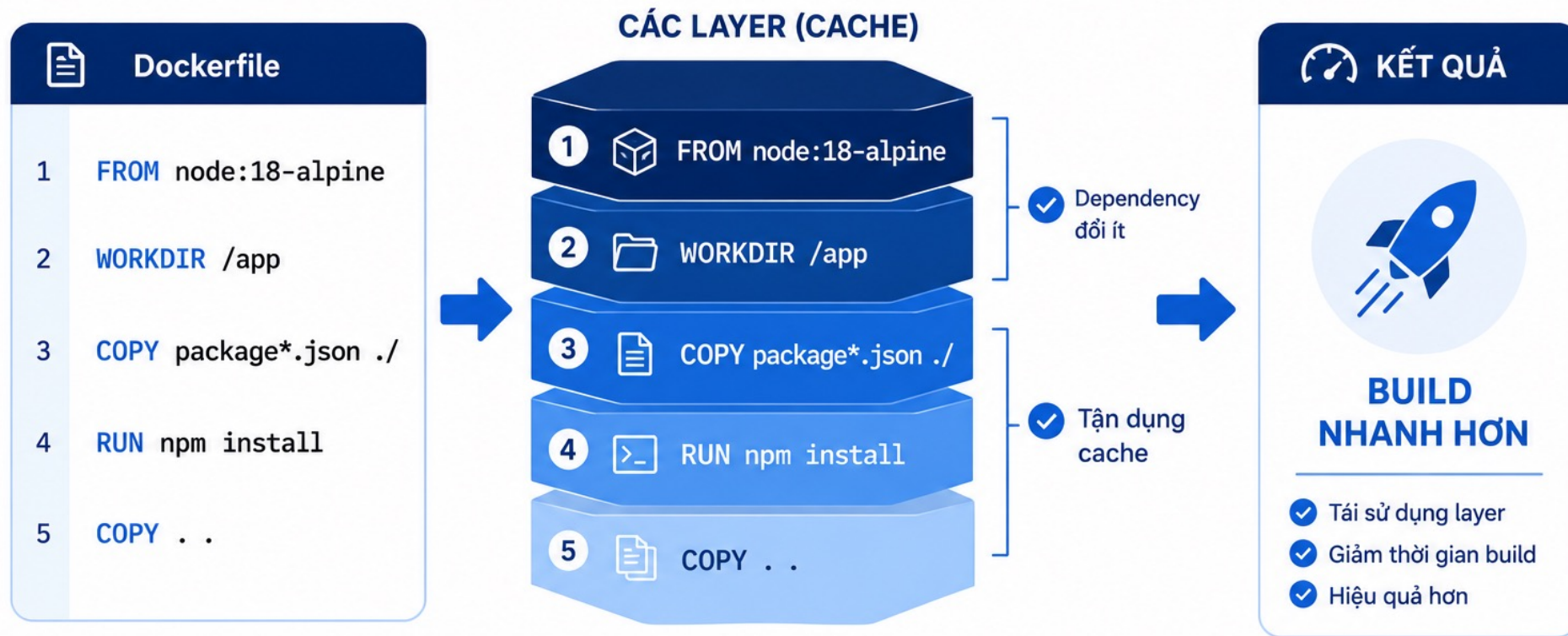
6

CMD



lệnh chạy app

DOCKER LAYER & CACHE



Viết Dockerfile



Build theo layer



Tận dụng cache



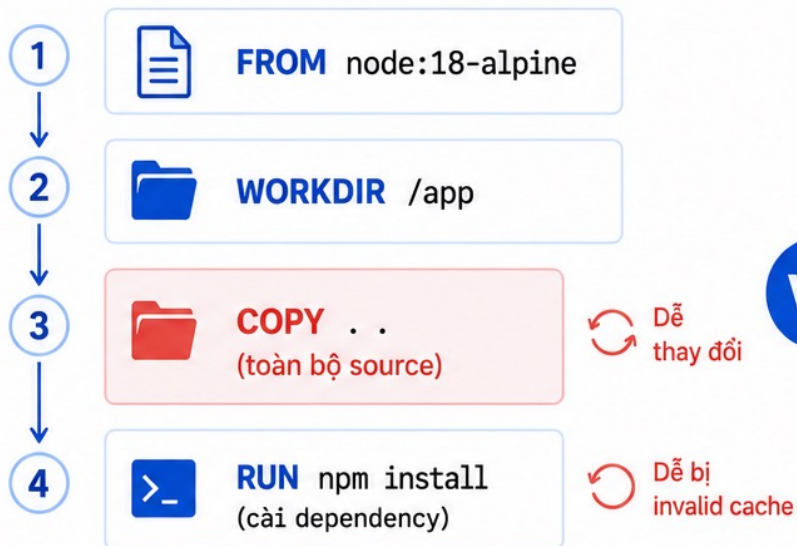
Build nhanh hơn





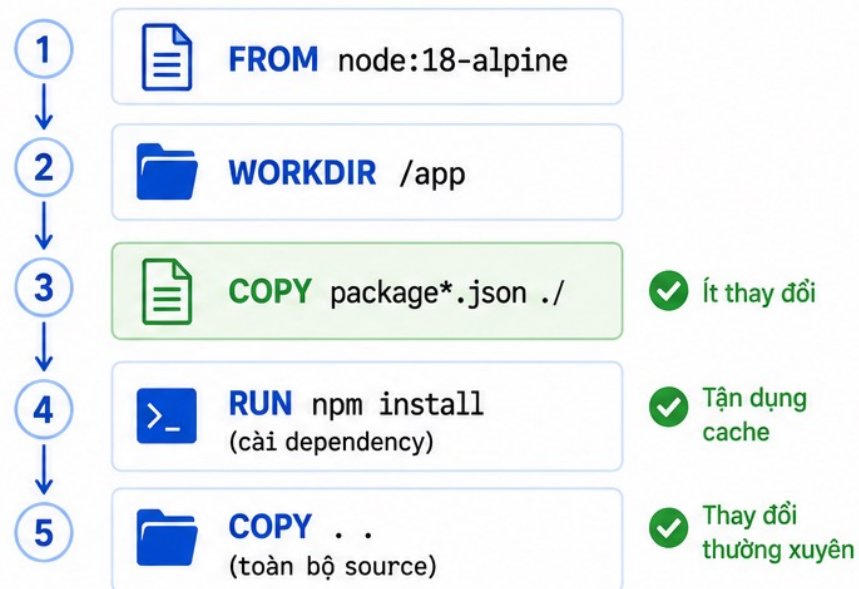
VÌ SAO COPY package*.json TRƯỚC?

❌ CÁCH LÀM CHƯA TỐI ƯU



 **BUILD CHẬM**
❌ Không tận dụng cache

✅ CÁCH TỐI ƯU



 **BUILD NHANH**
✅ Tận dụng cache
✅ Ít build lại dependency

 **NHANH HƠN**



MULTI-STAGE BUILD

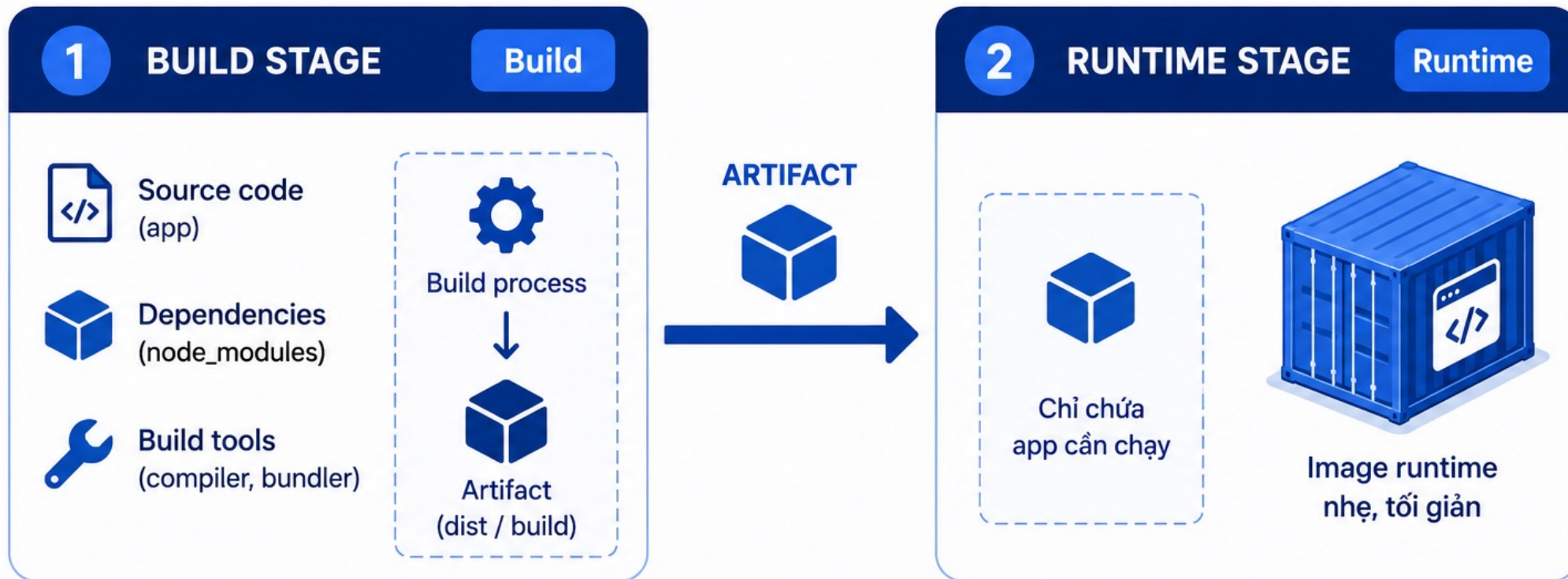


IMAGE NHẸ HƠN

Loại bỏ công cụ build và dependency thừa



SẠCH HƠN

Image nhỏ gọn, dễ bảo trì



AN TOÀN HƠN

Giảm bề mặt tấn công, ít rủi ro hơn

VÌ SAO NÊN DÙNG NON-ROOT USER?

 **CHẠY BẰNG ROOT**

**Root: rủi ro cao**

 Quyền quá lớn

 Nếu bị khai thác → chiếm toàn bộ hệ thống

 Có thể đọc/ghi mọi thứ

**USER: root**

VS

 **CHẠY BẰNG APPUSER**

**User thường: an toàn hơn**

 Quyền giới hạn

 Giảm mức ảnh hưởng khi có lỗi

 Chỉ truy cập được phần cần thiết

**USER: appuser**



GIẢM MỨC ẢNH HƯỞNG KHI CÓ LỖI



AN TOÀN HƠN



THỰC HÀNH TỐT

HEALTHCHECK TRONG DOCKER



STARTING

khởi động



HEALTHY

healthy / OK



UNHEALTHY

unhealthy / lỗi



Container chạy
chưa chắc app sống



Cần **HEALTHCHECK**
để giám sát sức khỏe
ứng dụng



Kiểm tra
định kỳ



Health Endpoint



HEALTHCHECK → kiểm tra sức khỏe ứng dụng

```
HEALTHCHECK CMD curl -f http://localhost/health || exit 1
```



Phát hiện sớm sự cố

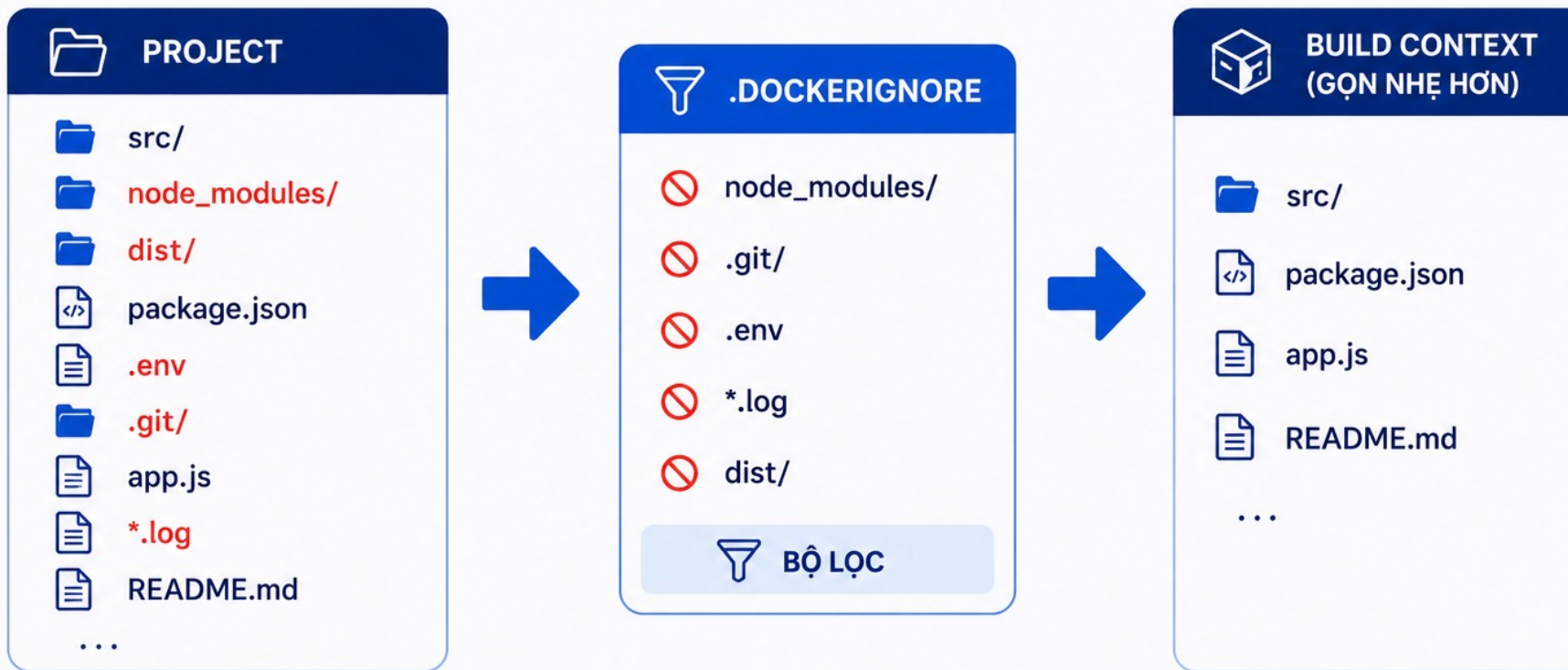


Tự động xử lý
(restart / thay thế)



Tăng độ tin cậy
cho hệ thống

.DOCKERIGNORE ĐỂ LÀM GÌ?



LỢI ÍCH



NHẸ HƠN

Giảm kích thước build context



NHANH HƠN

Tăng tốc quá trình build & push



AN TOÀN HƠN

Không đưa file nhạy cảm vào image



.env KHÁC .env.example

.env 



-  dữ liệu thật
-  có secret
-  không commit

VS

.env.example 



-  dữ liệu mẫu
-  hướng dẫn cấu hình
-  được push lên Git



BẢO MẬT

.env



Giữ riêng



.env.example

Chia sẻ



CHIA SẺ
AN TOÀN



ARG vs ENV

ARG



-  build-time
-  dùng khi build
-  không tồn tại khi chạy

VS

ENV



-  run-time
-  tồn tại trong container
-  dùng cho cấu hình app



ARG NODE_VERSION

```
docker build --build-arg NODE_VERSION=18 .
```



ENV PORT=3000

```
docker run -e PORT=3000 myapp
```



TÓM TẮT



ARG: chỉ dùng khi build



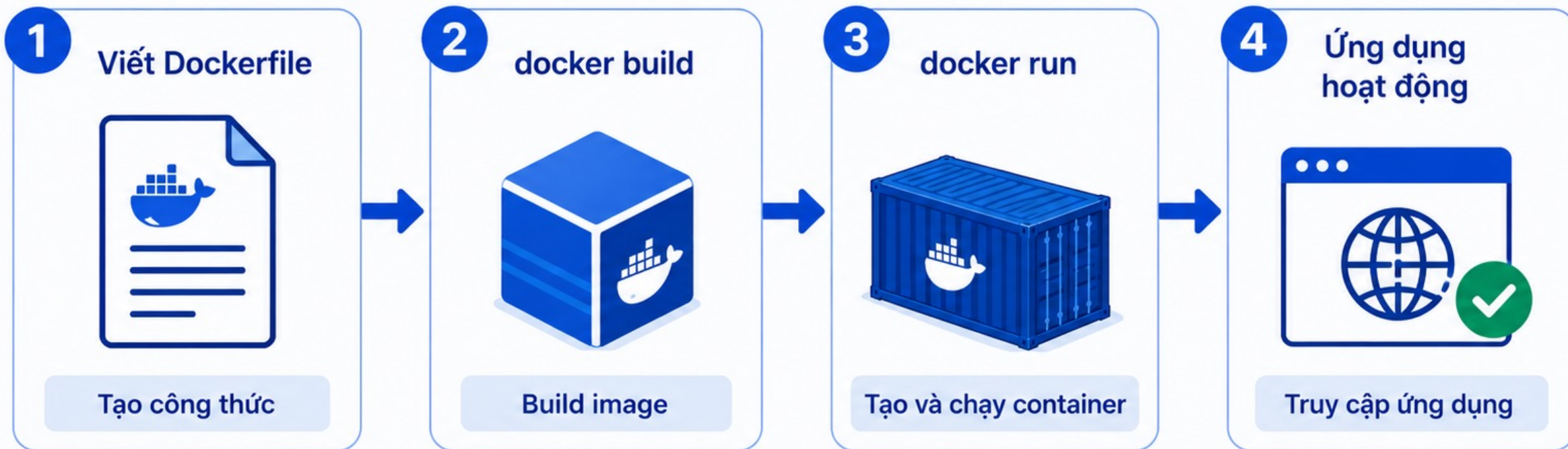
ENV: tồn tại khi chạy container



ENV thường dùng cho cấu hình app



BUILD & RUN CONTAINER



```
docker build -t my-app:v1 .
```

Build image từ Dockerfile



```
docker run -p 3000:3000 my-app:v1
```

Chạy container, map cổng 3000



Dockerfile



Image



Container



Ứng dụng hoạt động



SAU KHI CHẠY CONTAINER, KIỂM TRA GÌ?

1



`docker ps`



container
có chạy không

2



`docker logs`



log có
lỗi không

3



`curl /health`



app còn
phản hồi không

4



`docker inspect`



health status
ra sao

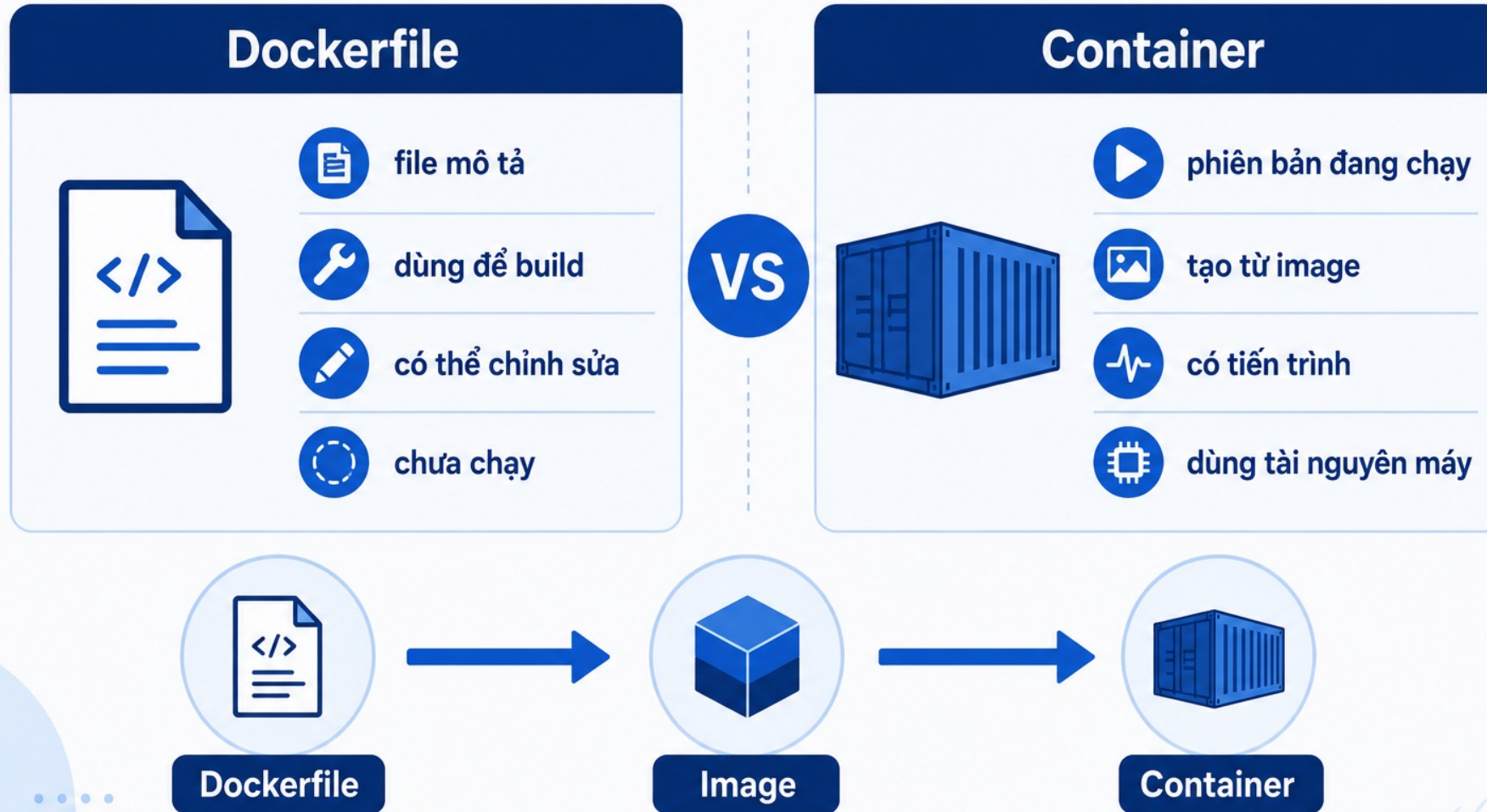


MỤC TIÊU:



Đảm bảo container chạy ổn định và ứng dụng hoạt động bình thường

DOCKERFILE KHÁC CONTAINER NHƯ THẾ NÀO?



DOCKER REGISTRY & TAG



Chia sẻ để triển khai trên mọi môi trường



TAG IMAGE NHƯ THẾ NÀO?

1. SEMANTIC VERSION



v1.0.0

2. GIT SHA



8a3b2c9

3. MÔI TRƯỜNG



latest / stable / staging



QUY ƯỚC LỚP:

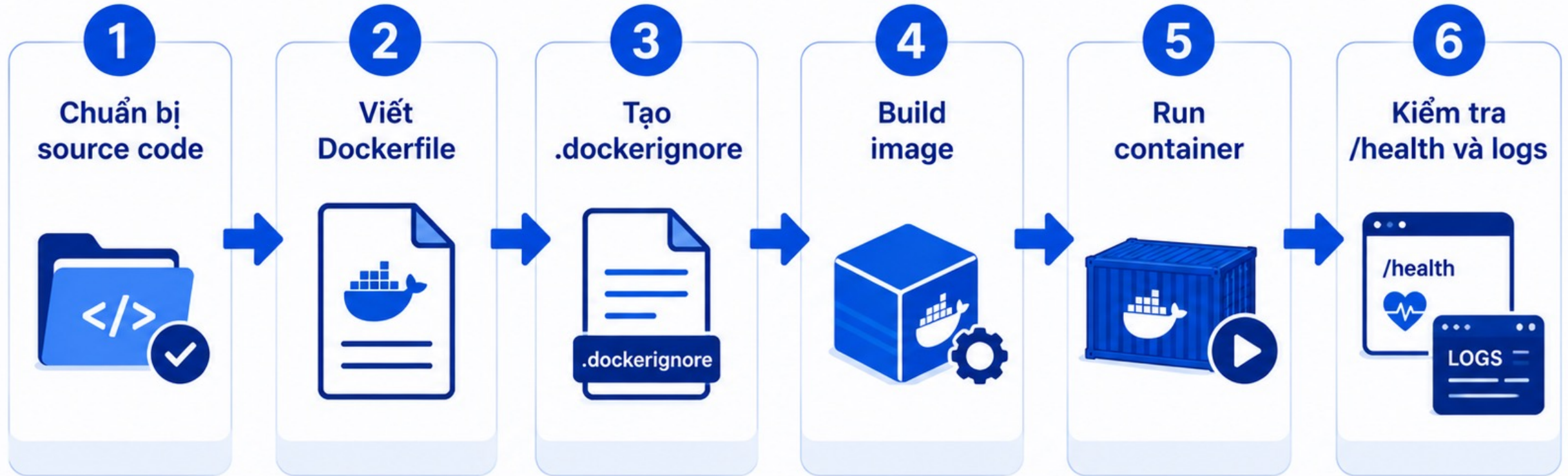
v0.1.0-*<team>*







QUY TRÌNH LAB DOCKER



MỤC TIÊU:

Build, chạy và kiểm tra ứng dụng bằng Docker





DEFINITION OF DONE



Build
thành công



Container
chạy được



/health
hoạt động



Có
.dockerignore



Có
.env.example



Có
non-root user



Có
HEALTHCHECK



Có
RUN_LOCAL.md



Push image
đúng tag



MỤC TIÊU:

Ứng dụng đóng gói sẵn sàng – Kiểm tra đầy đủ – Dễ triển khai





NHÓM CẦN NỘP NHỮNG GÌ?

1



Dockerfile

2



.dockerignore

3



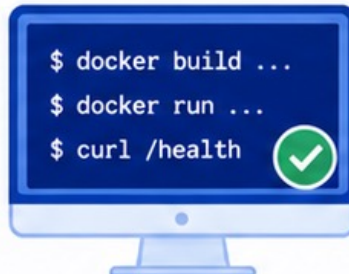
.env.example

4



RUN_LOCAL.md

5



ảnh/log build-run-test

6

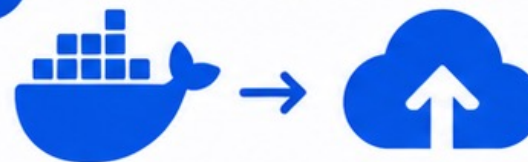


image đã push



LƯU Ý:



Push đúng tag theo quy ước lớp



Không commit file .env thật



MỘT SERVICE THÌ ĐƯỢC, NHIỀU SERVICE THÌ SÀO?

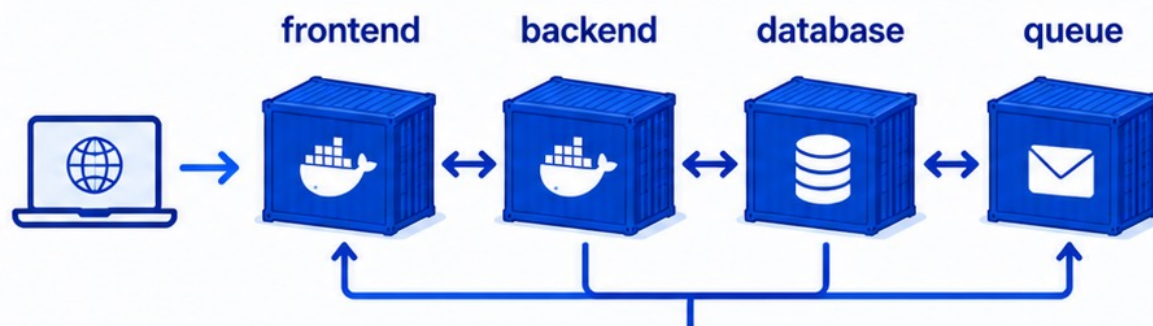
1 service



✓ dễ quản lý

VS

nhiều service



⚙ cần orchestration cơ bản



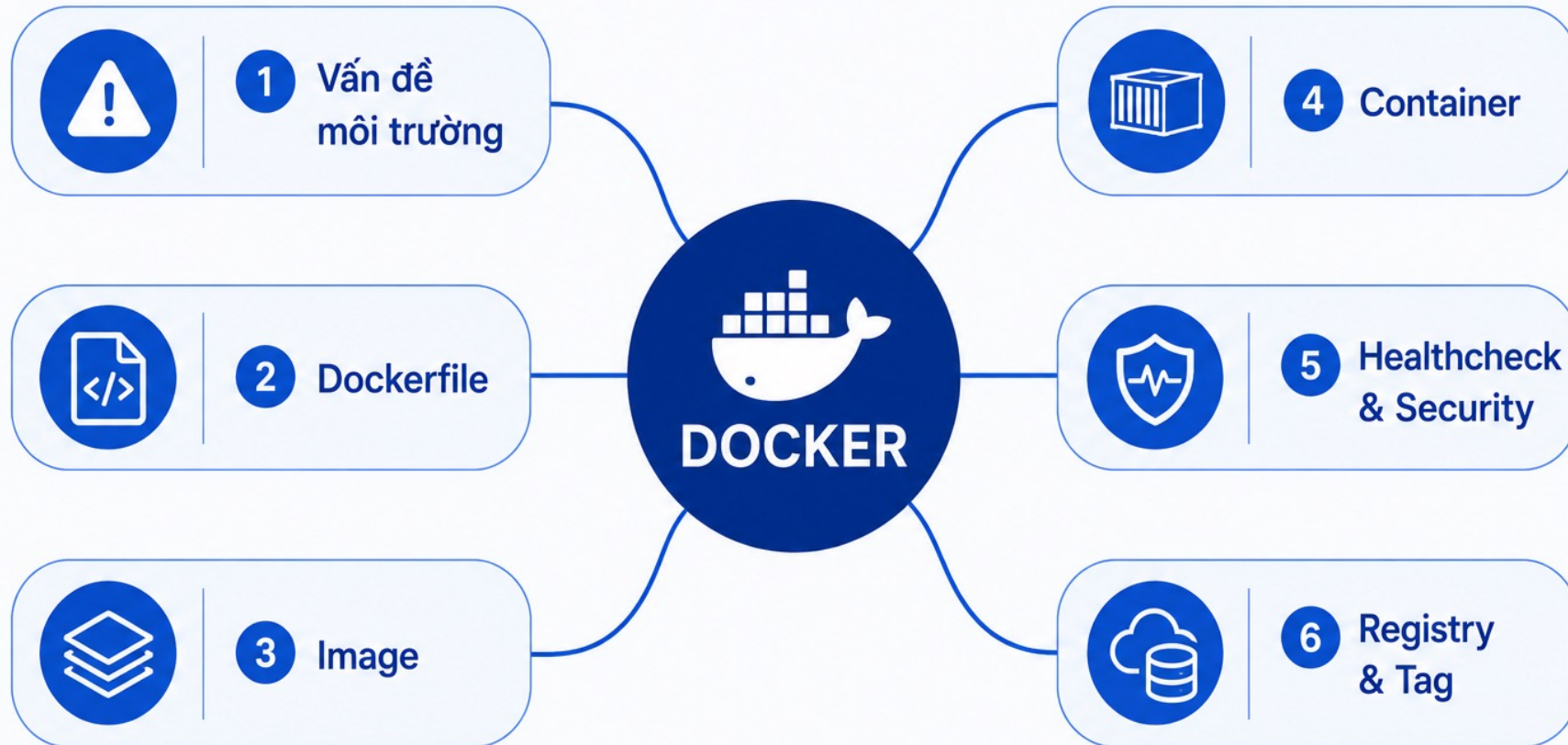
BUỔI SAU:



Docker Compose



TỔNG KẾT BÀI 4 - DOCKER





Thank You!

DAINAM
UNIVERSITY